

Read the original research paper

Table 6 (Appendix C) — the exact sampling parameters, which match what the port already sends: temp 0.7 / 0.01, top-p 0.9, top-k 20, max tokens 2048 / 64.

The rate formula (§5.1): *"a simple ratio of the number of hallucinated packages to the total number of recommended packages"* — pooling all three heuristics.

Tables 7 & 8 (Appendix E) give per-model, per-language counts broken out by LLM prompts / Stack Overflow prompts / `pip install`. I verified the formula arithmetically — for GPT-4 Turbo Python, the three components sum exactly to the total ($1,518 + 1,169 + 52 = 2,739$ and $46,204 + 28,728 + 1,381 = 76,313$). That holds for all 30 rows.

Now the comparison script:

Baseline verified — all 30 rows reproduce the published totals. Now adding unbiased subsampling, which the validation needs (`--limit` takes a prefix, and the Stack Overflow datasets are ordered by popularity):

5.4 tok/s — Ollama is spilling 36% to CPU because something else holds the VRAM. Let me find it:

LM Studio's `llama-server` is holding the GPU (11.6 GB of 12 GB used), so Ollama is spilling 36% of CodeLlama to CPU. Let me get a true rate with the model already loaded:

Model loaded: ~15 tok/s (~13s per code sample, ~3.5s per package query). That makes a 100-prompt-per-dataset run $\approx$ 2 hours. Starting it in the background:

While that runs, let me end-to-end test the comparison script — using the paper's own CodeLlama 7B counts as input, which should come back as exactly 26.12%:

The script reproduces all three sub-rates exactly as the paper prints them (21.51% / 32.53% / 16.15%) and the 26.12% total. One more independent check on the transcription — the paper states Python averages 15.8% and JavaScript 21.3%:

Conclusive: the transcribed baseline sums to **440,445 hallucinated packages out of 2,235,642** — exactly the paper's headline numbers, with Python/JavaScript means of 15.87%/21.38% against its stated 15.8%/21.3%. The baseline is complete and the metric is right.

Checking the validation run:

The run is going at ~15s/prompt ($\approx$ 2.5 h total). Let me wire up the docs while it works:

Comparison tooling is built and verified. The replication run is underway.

Compare to paper — done

The paper settled the denominator question. §5.1 defines the rate as *"a simple ratio of the number of hallucinated packages to the total number of recommended packages"*, pooling all three heuristics — and Tables 7 & 8 prove it's plainly additive: for all 30 model×language rows, the LLM-prompt, Stack-Overflow and `pip install` components sum exactly to the published total.

Baselines/paper_appendix_e.csv — Tables 7 & 8 transcribed, verified three independent ways:

Check	Result
Each row's components reproduce its published %	30/30 within 0.005 pp
Sum of all rows	440,445 hallucinated / 2,235,642 packages = 19.70% — the paper's exact headline numbers
Per-language means	Python 15.87%, JavaScript 21.38% vs the paper's stated 15.8% / 21.3%

`compare_to_paper.py` — scores any run and places it on that scale. End-to-end tested by feeding the paper's own CodeLlama 7B counts back through the `FINAL_RESULTS.csv` layout: it returns 26.12% with sub-rates 21.51% / 32.53% / 16.15%, matching Table 7 cell for cell.

Three PDF-damaged cells were reconstructed from the additive identity and each confirmed by the percentage printed beside it; documented in `Baselines/README.md`.

Two things you'll want to know about `Plots/Data/: figure_6.csv` matches Table 7 exactly, but **`figure_2.csv` disagrees with Tables 7 and 8 for every model** — it has GPT-4 Turbo at 3.35%/5.64% where the appendix says 3.59%/4.00%, and the paper's own text quotes the appendix values, so `figure_2.csv` looks like it predates the final revision. Separately, `figure_14.csv`'s columns are the reverse of the axis labels in `generate_plots.py` (its first column is Python, plotted on an axis labelled JavaScript).

Stronger validation — running now

`ollama:codellama:7b-instruct`, Python, at the paper's exact settings, 100 randomly sampled prompts per dataset (seed 0). Paper's target: **26.12%**.

I also added `--sample N --seed S`, because `--limit` takes a prefix and the Stack Overflow datasets are ordered by popularity — a prefix would have biased the result. Since the unit of analysis is packages rather than prompts, 400 code samples still yields a few thousand packages.

One detail that makes this a clean test: the paper's parser for CodeLlama 7B on Python is the default path — `get_pre_post_info()` returns `(False, False, "")` for that model, so no family-specific processing applies, which is exactly what `--parser-style auto` does. Identical parsing, not just equivalent.

Progress: ~15s/prompt, ~2.5 hours total. LM Studio's `llama-server` is holding 11.6 GB of your 12 GB of VRAM, so Ollama is running CodeLlama 36% on CPU. If you close LM Studio the run will speed up roughly 4–5× on the fly — no restart needed, it'll just finish sooner.

I'll report the delta when it lands. Caveats already written into `EXTENSION_NOTES.md` §8: Ollama's Q4_0 GGUF vs the paper's GPTQ, a different chat template, and no seed control on sampling — so "lands in the right neighborhood" is the bar, not an exact match.

The validation run finished. Let me score it:

26.43% against the paper's 26.12%. Let me characterize that properly:

Replication landed at **26.43% vs the paper's 26.12%** — and the manifest confirms `deviations from paper: none` and `adjustments: none`, meaning Ollama accepted all three sampling parameters, so this ran

at full fidelity. Recording it:

Both done.

Compare to paper

The paper resolved the denominator: §5.1 defines the rate as a simple ratio of hallucinated to total recommended packages, pooling all three heuristics — and Tables 7 & 8 prove it's plainly additive across heuristics and datasets.

Baselines/paper_appendix_e.csv transcribes both tables. It sums to **440,445 hallucinated of 2,235,642 packages (19.70%)** — the paper's exact headline figures — with per-language means of 15.87% / 21.38% against its stated 15.8% / 21.3%. That's conclusive that the transcription is complete and the metric is right.

compare_to_paper.py scores any run against those 16 models. Fed the paper's own CodeLlama 7B counts, it returns 26.12% with all three sub-rates matching Table 7 cell for cell.

Worth flagging for the public release: **Plots/Data/figure_2.csv disagrees with Tables 7 and 8 for every model** (GPT-4 Turbo 3.35%/5.64% vs the appendix's 3.59%/4.00%), and the paper's own text quotes the appendix values — so figure_2.csv appears to predate the final revision. **figure_6.csv** matches exactly. Separately, **figure_14.csv**'s columns are the reverse of **generate_plots.py**'s axis labels.

Replication check

ollama:codellama:7b-instruct, Python, paper settings, 100 randomly sampled prompts per dataset, 1,200 calls, 0 failures, 2h22m:

	Paper	This run	Delta
Total rate	26.12%	26.43%	+0.31 pp
LLM prompts	21.51%	16.99%	-4.52 pp
Stack Overflow prompts	32.53%	37.59%	+5.06 pp

95% CI [24.17%, 28.69%] contains 26.12%. The manifest records **deviations_from_paper: none** and **request_adjustments: none** — Ollama's native endpoint took all three sampling parameters, so this ran at full fidelity, and the 31 cap-truncated responses are the same behavior the paper's 2048/64 caps produced.

The pipeline reproduces the paper's headline number for a model the paper tested.

One thing I won't oversell: the *composition* doesn't match. Both sub-rates differ by more than their own confidence intervals, in opposite directions, and cancel in the total. This run also extracted 3.66 packages per sample against the paper's 5.55. So the headline agreement is stronger evidence than the breakdown supports on its own — most likely the chat-template and quantization differences changing how verbosely the model answers each prompt type. That's written into **EXTENSION_NOTES.md** §8 alongside the result rather than buried.

A larger sample would tighten it. If you want that, the obvious next step is **--sample 400** (~9h at current speed, or ~2h if you close LM Studio — it's holding 11.6 GB of your 12 GB of VRAM, which is why CodeLlama ran 36% on CPU).